INDEX

NAME R. Sri Yuvesh SUBJECT DAA

STD. _____ DIV. _____ ROLL NO. 2516O1267 SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Pr:1 - Finding Complexity using counter method	13/8	
2		Pr:- 2 - Finding Complexity using counter method.	13/8	
3		Pr:- 3 - Finding Complexity using Counter method.	13/8	
4		Pr :- 4 - Finding Complexity using Counter Method.	13/8	
5		Pr:- 5 - Finding Complexity using counter method.	13/8	
6		Divide and Conquer		
2.1		Number of zeroes in a given array	20/8	
2.2		majority Element.	20/8	
2.3		Finding Floor value	20/8	
2.4		Two elements Sum to X.	20/8	
2.5		Implementation of Quick Sort.	20/8	
		Greedy Algorithm		
3.1		Gr - Coin problem	10/9	
3.2		Gr - Cookie problem	10/9	

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
3.3		G-Burger Problem	10/9	✓
3.4		G-Array Sum Max problem	10/9	✓
3.5		G-Product of Array element - Minimum	10/9	✓
		Dynamic Programming		
4.1		playing with Number	17/9	✓
4.2		Playing with chessboard.	17/9	✓
4.3		Longest common Subsequence	17/9	✓
4.4		Longest Non Decreasing Subsequence.	17/9	✓
		competitive Programming		
5.1		Finding Duplicate $O(n^2)$.	24/9	✓
5.2		Finding Duplicate $O(n)$.	24/9	✓
5.3		Print Intersection of 2 sorted Array $O(m*n)$	24/9	✓
5.4		Print Intersection of 2 Sorted Array $O(m+n)$	24/9	✓
5.5		Pair with Difference $O(n^2)$.	24/9	✓
5.6		Pair with Difference $O(n)$	24/9	✓

Completed ✓

Problem 1: Finding Complexity using Counter method

Aim:

To convert the given algorithms # into a C program and determine the time capacity during the counter method

Algorithm:

1. Start the program
2. Read the positive integer n
3. ~~Initialize the positive integer n~~
3. Initialize i=1, s=1, count=3.
4. Repeat while s<=n:
   - Increment count
   - Increment i
   - Increment count
   - Add i to s
   - Implement ~~count~~ Count
   - Print the value of count
   - Stop the program.

Result:

~~The program's~~ time complexity is verified using ~~Counter~~ method.

Program

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int i=1;
    int s=1;
    count=2
    while(s<=n){
        count++;
        i++;
        count++;
        s+=i;
        count++;
    }
    printf("%d",count);
    return 0;
}
```

Output:

Input	Expected	output
2	12	12
1000	5002	5002
143	717	713

Input	Expected	Got
9	12	12
4	9	9

Program

```
#include <stdio.h>
int main(){
  int n;
  int count=1;
  scanf("%d", &n);
  if (n==1){
    count++;
  }
  else {
   count++;
   for (int i=1; i<=n; i++){
      count++;
      for (int j=1; j<=n; j++){
          count++;
          count++;
          count++;
          break;
       }
       count++;
    }count++;
  }
  printf("%d", count);
  return 0;
}
```

Output:

Input	Expected	Got
2	12	12
1000	5002	5002
143	717	717

Program 2: Finding complexity using counter method.

AIM:

To convert the given algorithm into a C program and determine the time complexity using the counter method.

ALGORITHM:

1. Start the program
2. Declare integer variables n and count
3. Read the value of n.
4. Initialize count = 0
5. If n==1
   - Increment count thrice
6. Otherwise
   - Increment count
   - Execute the outerloop from i=1 to 4
   - For each iteration
     - Increment count.
     - Execute the inner loop
     - Increment count three times inside the loop.
     - Break the inner loop after its first iteration.
   - Increment count.
   - Increment count after the outer loop.
7. Print count.
8. Stop the program.

Program 3: Finding complexity using counter method

Aim :-

To convert the given algorithm into a C program and determine the time complexity of it using counter method

Algorithm :-

1. Start the program.
2. Read the integer.
3. Initialize count = 0
4. Repeat the loop for i = 1 to n
   - Increment count
   - Increment count
   - check whether n % i == 0
   - If true, increment count.
5. After the loop, increment count.
6. Print the value of count
7. Stop the program.

Result :-

The program's time complexity using counter method.

```
main () {
int n:
scanf ("%d", &n);
int count = 0;
for (int i=0 ; i<=n; i++) {
    Count ++ ;
    Count ++ ;
    if (n-1.i == 0){
        count ++
    }
} Count ++;
printf ("%d", Count);
}
```

output

Input	Expected	Got
12	31	31
25	54	54
4	12	12

Program 4 : Finding complexity using counter method

Aim :-

To convert the given program into C program and determine the time complexity

Algorithm :

1. Start.
2. Read the integer
3. ~~Int~~ Initialize counter c=1
4. Execute the outer loop
5. For each iteration of outer loop
   - Increment c
   - Execute the second loop.
6. For each iteration of second loop
   - Increment c
   - Execute third loop and increment c
   - Increment c.
7. Increment c
8. Print the value of c
9. Stop the program.

Result :

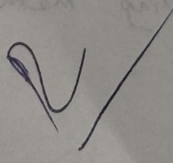
The program's time complexity is verified using ~~count~~ counting method.

Program

```
#include <stdio.h>
int main(){
    int n;
    int c=1;
    scanf("%d", &n);
    for(int i= n/2 ; i<n ; i++){
        c++;
        c++;
        for(int j=1 ; j< n; j++){
            c++;
            c++;
            for(int k=1; k<n; k++){
                c++;
                c++;
            }
        }
    }
    printf("%d", c);
}
```

Output

Input	Expected	Got
4	30	30
10	212	212



Program 5 : Finding complexity using counter method

Aim :

To convert the given program into C program and determine the time complexity.

Algorithm :

1. Start the program
2. Read the integer n
3. Initialize count
4. Increment count for every operation
5. Increment count using starting & ending of n for loop.
6. Print the value of count.
7. Stop the program.

Result :

The program's time complexity is verified using counter method.

Program

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d",&n);
    int count = 1;
    int rev = 0;
    count++;
    int remainder;
    count++;
    while (n != 0) {
        count++;
        rev = rev * 10 + remainder;
        count++;
        n /= 10;
        count++;
    }
    printf("%d", count);
}
```

Output

Input	Expected	Got
12	11	11
1234	19	19

Divide and conquer

1. number of zeroes in a given array ~~of 1's and 0's~~

Aim:

To write a program using divide and conquer to count the number of zeroes in the given array.

Algorithm:

```
Algorithm numzero(){
  read n, array of n elements (,
  low = 1, high = n, first zero = -1,
  WHILE low <= high, then
      mid = (low + high) / 2;
      if array[mid] == 0
          first zero = mid
          high = mid - 1;
      else
          low = mid + 1;   ENDIF
  ENDWHILE
  IF ( first zero == -1)
      print 0

  ELSE
      print n - first zero
  ENDIF
END
```

Program:

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    int low=0, high=n-1, firstzero=-1;
while (low<=high){
    int mid=(low+high)/2;
    if (arr[mid]==0){
        firstzero=mid;
        high=mid-1;
    }
    else {
      low=mid+1;
    }
}
if (firstzero==-1) {
    printf("%d", e);
}
else {
    printf("%d", n-firstzero);
}
return 0;
}
```

Sample test cases:

Input	Expected	Got
5 1 1 1 0 0	2	2
10 1 1 1 1 1 1 1 1 1 1	0	0
8 0 0 0 0 0 0 0 0	8	8

Result:

The program to find number of zeroes using divide & conquer was written and executed successfully.

Problem 2: Majority Element

Aim: To find the majority elements in a array of size n

Algorithm:

```
Algorithm major ( arr, l, r)
If l=r
   return Arr[l]
mid = (l+r)/2
leftmajor ← major (arr, l, m)
rightmajor ← major (arr, m+1, r)
If leftmajor = rightmajor
      return leftmajor
leftcount ← 0
rightcount ←0
for i ← l to r
   if arr[i]= leftmajor then
      leftcount ← leftcount +1
   if arr[i] = rightmajor then
      rightcount ← rightcount +1
if leftcount > rightcount
      return leftcount;
else
      return rightcount;
```

Program:

```
#include <stdio.h>
int major(int arr[], int l, int r){
    if (l==r) return arr[l];
    int m = (l+r)/2;
    int left = major(arr, l, m);
    int right = major(arr, m+1, r);
    if (left == right) return left;
     int lc = 0, rc = 0;
    for(int i=l; i<=r; i++){
        if (arr[i] == left) lc++;
        if (arr[i] == right) rc++;
    }
     return (lc > rc) ? left : right;
}
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    printf("%d", major(arr, 0, n-1));
    return 0;
}
```

Sample test cases:

Input	Expected	Got
3 3 2 3	3	3

Result:

Hence the program find the majority element was written and executed sucessfully.

Problem 3: Finding Floor

Aim:

To write a program find the floor value of a number in an array using divide and conquer

Algorithm:

```
Algorithm FindFloor() {
    Read array of size : n
    Read value x
    low ← 0
    high ← n-1
    floor ← -1
    while low ≤ high
     mid ← (low + high)/2;
     if arr[mid] = x then
     Floor ← arr[mid]
     break
    else if arr[mid] < x then
          Floor ← arr[mid]
          low ← mid +1
    else
         high ← mid -1
    end while
    print floor.
```

Program:

```
#include <stdio.h>
int main(){
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    int low=0, high=n-1;
    int floor=-1; scanf("%d", &x);
    while(low<=high){
        int mid=(low+high)/2
    if(arr[mid]==x){
        floor=arr[mid];
        break;
    else if(arr[mid]<x){
        floor=arr[mid];
        low=mid+1;
    }
    else{
        high=mid-1;
    }
    }
    printf("%d", floor);
    return 0;
}
```

Sample test cases:-

Input	Expected	Got
6 1 2 8 10 12 19 5	2	2
5 10 22 85 108 129 100	85	85

Result:

Hence the program to find at the position value of the target in an sorted array was written and executed successfully.

Problem 4: Two elements sum to x

Aim:

To find whether if there exist two elements in a sorted array whose sum is equal to a given value of x using Divide and conquer method.

Algorithm:

Algorithm find two sum() {

read n

read array arr [0 ..... n-1].

Read x

Set left ← 0

Set right ← n-1

while left < right

Sum ← arr [left] + arr [right]

if sum = x then

print : arr [left]

print : arr [right]

stop

else if sum < x then

left ← ~~right~~ left + 1

else

right ← right - 1

End while

Print "No"

stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, x;
    scanf("%d", &n);
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i<n; i++) scanf("%d", &arr[i]);
    scanf("%d", &x);
    int i=0, j = n-1;
    while (i<1) {
        int sum = arr[i] + arr[j];
        if (sum == x) {
            printf("%d\n%d\n", arr[i], arr[j]);
            free(arr);
            return 0;
        }
        else if (sum < x) {
            i++;
        }
        else {
            i--;
        }
    }
    printf("no");
    free(arr);
    return 0;
}
```

Sample test cases:

Input	Expected	Got
4 2 4 8 18 14	4 14	4 18
5 2 4 6 8 10 100	NO	NO

Result:

Thus, the program successfully checks whether the sum of two elements in the array is equal to target number.

Program 5: Implementation of quick sort.

Aim:

To sort a list of elements in ascending order using the quick sort algorithm

Algorithm:

Procedure quicksort(arr, low, high)

If low < high

pivot Index ← partition(arr, low, high)

quicksort(arr, low, pivotIndex-1)

quicksort(arr, Pivot index, high)

End if

End Procedure

Procedure partition(arr, low, high)

pivot ← arr[high]

i ← low - 1

for j ← low to high - 1

If arr[j] ≤ pivot then

i ← i+1

swap arr[i] and arr[j]

End if

End for

Swap arr[i+1] and arr[high]

return i+1

End Procedure

Start

read n

Read array elements

quicksort(arr, 0, n-1)

print array

STOP.

Program:

```
#include <stdio.h>
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) swap(&arr[++i], &arr[j]);
    }
    swap(&arr[i+1], &arr[high]);
    return i+1;
}
void quicksort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++) scanf("%d", &arr[i]);
    quicksort(arr, 0, n-1);
    for (int i=0; i<n; i++) printf("%d ", arr[i]);
    return 0;
}
```

Sample Test cases:

Input	Expected	Got
5 6 7 3 4 1 2 9 8 7 8 1 1 9 0 1 0 1 5 6 7 8 9 0 3 2 5 6 1 1 1 0 9 0 1 1 4	1 2 3 4 6 7 7 8 9 8 1 1 0 1 1 3 2 5 6 7 8 9 0 9 0 1 1 4	1 2 3 4 6 7 7 8 9 8
12 9 8 7 6 5 4 3 2 1 1 0 1 1 9 0	1 2 3 4 5 6 7 8 9 8 1 0 1 1 9 0	1 2 3 4 5 6 7 8 9 8 1 0 1 1 9 0

Greedy Algorithms:

Problem 1: Coin problem

Aim:-
To write a program to find the minimum number of coins/notes required to make change for give value V.

Algorithm:

```
Algorithm findcount(){
    Start
    Read the value V from user
    initialize denominations[] = {1000, 500, 100, 50, 20, 5, 2, 1}
    initialize count = 0
    for d = 0 to 8:
        count = V / denominations[d]
        V = V mod denominations[d]
    print count
    stop
}
```

Program:

```
#include <stdio.h>
int main(){
    int v;
    scanf("%d", &v);
    int denominations[]: {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;
    for (int i=0; i<9; i++){
        count += v / denominations[i];
        v %= denominations[i];
    }
    printf("%d", count);
    return 0;
}
```

Sample test cases:

Input	Expected	Got
49	5	5

Result:

Thus, the program was written and executed successfully.

To write a program to maximize the number of content children, given each child's greed factor and cookie's size

Algorithm ():

```
Algorithm cookies (){
    Start
    Read n and greed factor array g [0 ..... n-1].
    Read m and cookie size array s [0 ..... m-1].
    initialize used [j] = 0 for all j;
    initialize count = 0
    for i ← 0 to n-1 Do:
        for j ← 0 to m-1 Do:
            If used [j] == 0 and s[i] >= g[i]
                Count ← count + 1
                used [j] = 1
                break.
        print count
        Stop
}
```

Program:

```
#include <stdio.h>
int main(){
 int n, m;
 scanf("%d", &n);
 int g[n];
 for (int i=0; i<n; i++) scanf("%d", &g[i]);
 scanf("%d", &m);
 int s[m], used[m];
 for (int i=0; i<m; i++) { scanf("%d", &s[i]);
        use[i]=0;
 }
 int count = 0
 for (int i=0; i<n; i++){
    for (int j=0; j<m; j++){
       if (used[j]==0; && s[j]>=g[i]){
          count++;
          used[i]=1;
          break;
       }
    }
 }
 printf("%d", count);
 return 0;
}
```

Sample Test cases :

Input	Expected	Got
2 1 2 3 123	2	2

Result :

Hence the program was written and executed successfully

Problem 3 : Burger Problem

Aim:

To write a program to determine the min. distance a person needs to run after eating a number burgers.

Algorithm:

Algorithm mincal () {

Start

Read n and calories array arr [0 .... n-1]

Sort the array in descending order

Initialize sum ← 0

For i ← 0 to i ← n-1

sum ← sum + (n * arr [i])

print sum.

Stop

}

Result:

Thus, the program was written and executed successfully.

Program:

```
#include <stdio.h>
int main(){
 int n;
#include <math.h>
int main(){
   int n;
   scanf("%d", &n);
   int arr[n];
   for(int i=0; i<n; i++) scanf("%d", &arr[i])
    int temp;
    for(int i=0; i<n; i++){
        for(int j=i+1; j<n; j++){
            if(arr[i]<arr[j]){
              temp=arr[i];
              arr[i]=arr[j];
              arr[j]=temp;
            }
        }
    }
    int sum=0
    for(int i=0; i<n; i++){
        sum+=(pow(n,i)*arr[i]);
    }
    printf("%d", sum);
    return 0;
}
```

Input	Expected	Got
3 1 3 2	18	18
4 7 4 9 6	389	389
3 5 1 0 7	76	76

Problem 4: Array Sum max Problem

Aim:

To write a program to maximize the sum of a[i] and its corresponding index.

Algorithm:

```
Algorithm maxsum() {
    Start
    Read n and array a[0.... n-1]
    Sort a[] in ascending order using any
    Initialize sum ← 0
    For i:0 to n-1 do
        sum ← sum + (a[i]*i)
    print sum
    stop
}
```

Result:

Thus the program was written and executed successfully.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp (const void * a, const void * b){
    return *(int *)a - *(int *)b;
}
int main (){
    int n;
    scanf ("%d", &n);
    int a[n];
    for (int i=0; i<n; i++) scanf ("%d", &a[i]);
    qsort (a, n, sizeof(int), cmp);
    int sum =0;
    for (int i=0; i<n; i++) sum += a[i]*i;
    printf ("%d", sum);
    return 0;
}
```

Sample test cases:

Input	Expected	Got
5 4 3 4 5	40	40
2 45 3	45	45

Program 5: Product of Array Elements - Minimize

Aim:

To write a program to arrange two array such that sum of pairwise products is minimized

Algorithm:

Algorithm minsum () {

start

read n, array A[0..... n-1] and B[0... n-1]

sort A in ascending order

sort B in descending order.

Initialize sum = 0

for i = 0 to n-1 do

sum ← sum + (A[i] * B[i])

print sum

stop

}

Result:

Thus the program was written and executed successfully

Program:

```
#include <stdio.h>
#include <stdlib.h>
int compare1(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
int compare2(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}
int main(){
    int n; sum=0
    scanf("%d", &n);
    int a[n], b[n];
    for(int i=0; i<n; i++) scanf("%d", &a[i]);
    for(int i=0; i<n; i++) scanf("%d", &b[i]);
    qsort(a, n, sizeof(int), compare1);
    qsort(b, n, sizeof(int), compare2);
    for(int i=0; i<n; i++){
        sum += (a[i] * b[i]);
    }
    printf("%d", sum);
    return 0;
}
```

sample test cases:

Input	Expected	Got
3 1 2 3 4 5 6	28	28
4 7 5 1 2 1 3 4 1	590	590

Dynamic

Problem 1: Playing with Numbers

Aim:

To find the total numbers of ways number can be represented by using 1 or 3.

Algorithm:

```
Algorithm numbers(){
    Start
    Read n
    set dp[0]=1
    for i<=1 to n
        dp[i] = dp[i-1]
    If i>=3
        dp[i] = dp[i] + dp[i-3]
    END IF
    END FOR
    print dp[n]
    Stop
}
```

Program:

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    long long dp[n+1];
     dp[0] = 1;
     for(int i=1; i<=n; i++){
          dp[i] = dp[i-1];
          if (i>=3)
              dp[i] += dp[i-3];
      }
      printf("%d", dp[n]);
      return 0;
}
```

Output:

Input	Expected	Got
6	6	6
25	8641	8641
100	2438281959 67	2438281959 67
	21629	21629

Result:

Thus, the program was successfully written and executed.

Problem 2: Playing with chessboard

Aim: To find the maximum monetary value path from top left cell to bottom right cell while moving only right or down.

Algorithm:

```
Algorithm maxpath()
  Start
  Read n
  Declare matrix a[n][n]
  FOR i ← 0 to n-1 Do
    FOR j ← 0 to n-1 DO
      Read a[i][j]
    END FOR.
  END FOR
  FOR j ← 1 to n-1 Do
      A[0][j] ← A[0][j] + A[0][j-1]
  END FOR
  FOR i ← 1 to n-1 Do
      A[i][0] ← A[i][0] + A[i-1][0]
  END FOR
   FOR i ← 1 to n-1 Do
        FOR j ← 1 to n-1 Do
        IF A[i-1][j] > A[i][j-1] THEN
            A[i][j] ← A[i][j] + A[i-1][j]
        ELSE
          A[i][j] ← A[i][j] + A[i][j-1]
        END IF
    END FOR
END FOR
END FOR print[n-1][n-1]
  END
```

Program:

```
#include <stdio.h>
int main (){
    int n;
    scanf ("%d", &n);
    int a[n][n];
    for (int i=0; i<n; i++){
        for (int j=0; j<n; j++){
            scanf ("%d", &a[i][j]);
        }
    }
    for (int i=0; i<n; i++){
        a[0][i] = a[0][i] + a[0][i-1];
    }
    for (int j=1; j<n; j++){
        a[i][0] = a[j][0] + a[j-1][0];
    }
    for (int i=1; i<n; i++){
        for (int j=1; j<n; j++){
            if (a[i][j] > a[i][j-1])
                a[i][j] += a[i-1];
            else {
                a[i][j] += a[i][j-1];
            }
        }
    }
    printf ("%d\n", a[n-1][n-1]);
}
```

Output:

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19
3 1 3 1 1 5 1 4 2 1	12	12
4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28

Result:

Thus the program to find the highest monetary path was written and executed successfully

Problem 3: Longest common Subsequence

Aim:

To find the length of longest subsequence between two strings.

Algorithm:

Algorithm Subsequence () {

Start

Read string S_1, S_2

n = length of S_1,

m = length of S_2,

Create dp [m+1][n+1].

Set first row and first column to 0

For i = 1 to n

For j = 1 to m

If $S_1[i-1] == S_2[j-1]$ then

dp [i] [j] = 1 + dp [i-1] [j-1]

ELSE

dp [i] [j] = maximum (dp [i-1] [j],
dp [i] [j-1])

END IF

END FOR

Print dp [n] [m]

Stop

}

Program:

```
#include <stdio.h>
#include <string.h>
int max(int a, int b){
    return (a>b) ? a : b;
}
int main(){
    char s1[100];
    char s2[100];
    scanf("%s", s1);
    scanf("%s", s2);
    int n = strlen(s1);
    int m = strlen(s2);
    int dp[n+1][m+1];
    for(int i=0; i<=n; i++){
        dp[i][0] = 0;
    }
    for(int j=0; j<=n; j++){
        dp[0][j] = 0;
    }
    for(int i=1; i<=n; i++){
        for(int j=1; j<=m; j++){
            if(s1[i-1] == s2[j-1]){
                dp[i][j] = 1 + dp[i-1][j-1];
            }
            else{
                dp[i][j] = max(dp[i-1], dp[i][j-1]);
            }
        }
    }
    printf("%d\n", dp[n][m]);
}
```

Output	Expected	Got
a a b	2	2
a 2 b		
ABCD	4	4
AB CD		

Result:

Thus, the program to find the longest substring between two strings was written and executed successfully.

Problem 4 : DP - Longest non-decreasing subsequence

Aim :

To find the length of longest non-decreasing subsequence in a given sequence.

Algorithm :

```
Algorithm subsequence()
  Start
  Read n
  declare array arr[n] and dp[n]
  for i<-0 to n-1 DO
      Read arr[i]
      dp[i]<-1
  END FOR.
  max<-1
  for i<-1 to n-1 DO
      for j<-0 to i-1 DO
          IF arr[j]<-arr[i] Then
              If dp[j]+1 > dp[i] then
                  dp[i] <- dp[j]+1;
              END IF
          END IF
      ENDFOR
      IF dp[i] > max then
          max<-dp[i]
      END IF
      END FOR
      print max
      Stop
}
```

Program:

```
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    int dp[n];
    for(int i=0;i<n;i++){
      scanf("%d",&arr[i]);
        dp[i]=1;
    }
    int mnc=1;
    for(int i=1;i<n;i++){
        for(int j=0;j<i-1;j++){
            if(arr[j]<=arr[i]){
                if(dp[j]+1>dp[i]){
                    dp[i]=dp[j]+1;
                }
            }
        }
        if(dp[i]>mnc){
            mnc=dp[i];
        }
    }
    printf("%d",mnc);
}
```

Output:-

Input	Expected	Got
9 -1 3 4 5 2 2 2 2 3	6	6
7 1 2 2 4 5 7 6	6	6

Result:

Thus the program to find the longest non-decreasing subsequence in a given sequence was written and executed successfully.

Competivity programming

1. Finding Duplicates (O(n^2) - Time , O(1) Space)

Aim:

To find the duplicate element present in an list of numbers.

Algorithm:

```
Algorithm duplicates()
    start
    Read n, declare arr[n]
    For i<-0 to n-1 Do
        Read arr[i]
    END FOR
    For i<-0 to n-1 Do
        Read arr[i] For j<-0 to n-1 Do
    End For  If(a^^(i)==arr(j) and i!=j)) then
    For i=   print (i)
                stop
            End if
        End For
    End Do
    End
}
```

Program:

```
#include <stdio.h>
int main(){
 int n; scanf("%d",&n); int arr[n];
 for(int i=0;i<n;i++){
    scanf("%d",&arr[i]);
 }
 for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
       if(arr[i]==arr[j] && i!=j){
          printf("%d",arr[i]);
          break;
       }
    }
 }
}
```

Output:

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 4 1 2 3 4	4	4

Result:

Thus, the program to find the duplicate element was written and executed successfully.

2. Finding Duplicate O(n)-Time

Aim:

To find the duplicate element in O(n) time.

Algorithm:

```
Algorithm duplicate(){
    Start
    Read n
    declare arr[n]
    for i<-0 to n-1
        Read arr[i]
    End for
    For i<-0 to n-1 Do
        For i<-0 to i. Do
            if arr[i]=arr[i] then
                print arr[i]
                Stop
            End if
        End for
    End for
Stop
}
```

Program:

```
#include <stdio.h>
int main(){
int n;
scanf("%d", &n);
int arr[n];
for(int i=0; i<n; i++) scanf("%d", &arr[i])
for(int i=0; i<n; i++){
    for(int j=0; j<i; j++){
        if(arr[i]==arr[j] && i!=j){
            printf("%d", arr[i]);
            return 0;
        }
    }
}
return 0;
}
```

Inputs:

Input	Expected	Got
11 10976512384 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4 1	1	1

Result:

Thus, the program was written and executed successfully

Problem 3: print intersection of 2 Sorted array.

Aim:

To print the intersection of 2 array in $O(m * n)$ time.

Algorithm:-

```
Algorithm intersection(){
    Start
    Read T
    while T > 0 Do
    Read m
    set arr[n]
    For i<-0 to n Do
       Read arr[i]
    End For
    Read m
    Declare arr2(m)
     For i<-0 to m-1
        Read arr2[i]
     End for
     set i<-0, j<-0, first<-1
     While i<n and j<m Do
        if arr[i] = arr[j] then
         If first ==0 Then
            Print Space
         End if
         Print arr[i]
         set first<-0
         i<- i+1
         j<- j+1
         If arr[i]<- arr[j] then.
         Else
            j<- j+1
         End if
         End while
         print newline
         T<- T-1
         End while
        End
}
```

Program

```
#include <stdio.h>
int main() {
    int t;
    scanf("%d", &t);
    while (t > 0) {
        int n, m;
        scanf("%d", &n);
        int arr[n];
        for (int i=0; i<n; i++) scanf("%d", &arr[i]);
        scanf("%d", &m);
        int arr2[m];
        for (int i=0; i<m; i++) scanf("%d", &arr2[i]);
        int i=0, j=0;
        int first=1;
        while (i<n && j<m) {
            if (arr[i] == arr2[j]) {
                if (first) printf(" ");
                printf("%d", arr[i]);
                first=0;
                i++;
                j++;
            } else if (arr[i] < arr2[j]) {
                i++;
            } else j++;
        }
        printf("\n");
        t--;
    }
    return 0;
}
```

Test cases:

Input	Expected	Got
1 3 101787 6 2 7 10 15 5 7 24	10 8 7	10 8 7

Result:

Thus the program was written and executed successfully.

Problem 4: Print Intersection of 2 sorted array

Aim:

To print the common elements in an sorted array in $O(m+n)$ time.

Algorithm:

```
Start
Read T
while T>0 do
  Read n1
  declare array a[n1]
  for i<-0 to n1-1 Do
      Read a[i]
  End for
  Read n2
  declare array b[n2]
  for i<-0 to n2-1 Do
    Read b[i]
  end for
  i<-0, j<-0
  while (i<n1 and j<n2) Do
      if a[i]=b[j] then
        print a[i] +" "
        i<-i+1
        j<-j+1
      else if a[i]<b[i] then
          i<-i+1
      else
          j<-j+1
      End if
    End while
    print newline
     T<-T-1
  End while
stop.
```

Program:

```
#include <stdio.h>
int main(){
    int t;
    scanf("%d",&t)
    while(t--){
        int n1,n2;
        scanf("%d",&n1);
        int a[n1];
        for(int i=0; i<n1; i++) scanf("%d",&a[i])
        scanf("%d",&n2);
        int i=0, j=0;
        while(i<n1 & j<n2){
            if(a[i]==b[j]){
                printf("%d", a[i]);
                i++, j++;
            }else if(a[i]<b[j]{ i++;
                else j++
        }
        printf("\n");
    }
    return 0;
}
```

Sample Test cases:

Input	Expected	Got
1 3 101757	1057	1057
6 2 7101551246		

Result:

Thus the program was written and executed successfully.

Problem 5: Pair with Difference : $O(n^2)$ - Time complexity.

Aim:

Given an array A of sorted integers and another non-negative int k, find if there exist 2 numbers in A such that A[j] - A[i] = k, i != j.

Algorithm:

```
Start
Read n
declare array a[n]
For i<-0 to n-1 Do
    Read a[i]
End for
Read k
i<-0, j<-1
while (i<n and j<n ) Do
~~if (a[j]-a[i]==k) then~~
if i==j then
   j= j+1
if (a[j]-a[i]=k) then
   print i
   exit
else if a[i]-a[i]<k then
        j+1
else
   j<-i+1
End if
End while
print 0
End.
```

Program:

```
#include <stdio.h>
int main(){
  int n, k;
  scanf("%d",&n);
  int a[n];
  for(int i=0; i<n; i++) scanf("%d",&a[i]);
  scanf("%d",&k);
  int i=0, j=1;
  while(i<n && j<n){
    if(i==j){
      j++
    }
    else if(a[j]-a[i]==k){
      printf("1"); return 0;
    }
    else if(a[j]-a[i]<k){
      j++;
    }else{
      i++;
    }
    printf("0");
    return 0;
}
```

Test case

Input	Expected	Got
3 1 3 5 4	1	1

Result:

The program was written and executed successfully.

Problem -6 Pair with Difference, O(n) - Time com

Aim:

To check if there exist 2 number in a sorted array of number where their difference is equal to target

Algorithm:

Start
Read n
declare array a[n]
for i<-0 to n-1 do
    Read a[i]
end for
i<-0, j<-1
while (i<n and j<n) Do
    if i=j Do
        j<-j+1
    else if (a[j]-a[i]=k) Do
        print "1"
        exit
    else if (a[j]-a[i]<k) Do
        j<-j+1
    else
        i<-i+1
End while
print "0"
End.

Program:

```
#include <stdio.h>
int main(){
  int n,k;
  scanf("%d",&n);
  int a[n];
  for(int i=0;i<n;i++) scanf("%d",&a[i]);
  scanf("%d",&k);
  int i=0,j=1;
  while(i<n && j<n){
    if(i==j){
      i++;
    else if(a[j]-a[i]==k){
      printf("1");
      return 0;
    }else if(a[j]-a[i]<k) j++;
    else i++;
  }
  printf("0");
  return 0;
}
```

Test case:

Input	Excepted	Got
3 1 3 5 4	1	1

Result:

The program was written and executed successfully.